**Arzel Andrei Israel**
**Lynxie Jagonob**
**John Walter Pediangco**
**John Christian Setubal**
**Emmanuel Siastres**

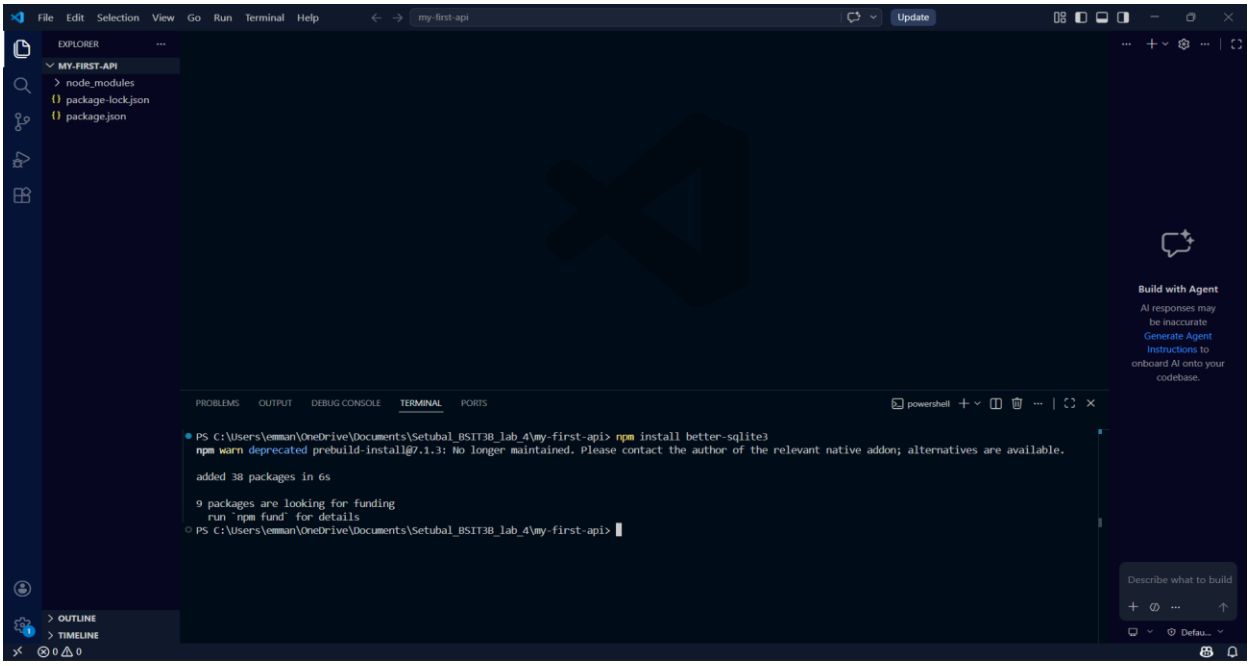
Lab 4. Database Connection (Data Persistence)

**Objective:** Transform your "volatile" API (which loses data on restart) into a **Persistent API**. You will use a local SQLite database to store product information.

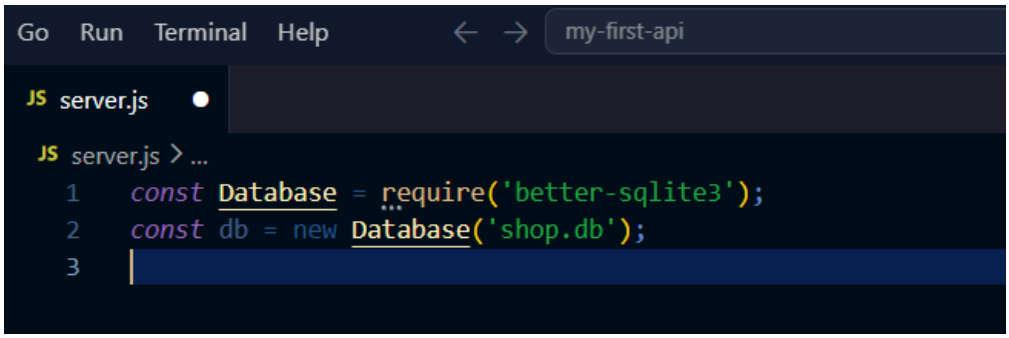
We'll set the setting for the server/site as E-commerce (hosting random products)

Setup Instructions

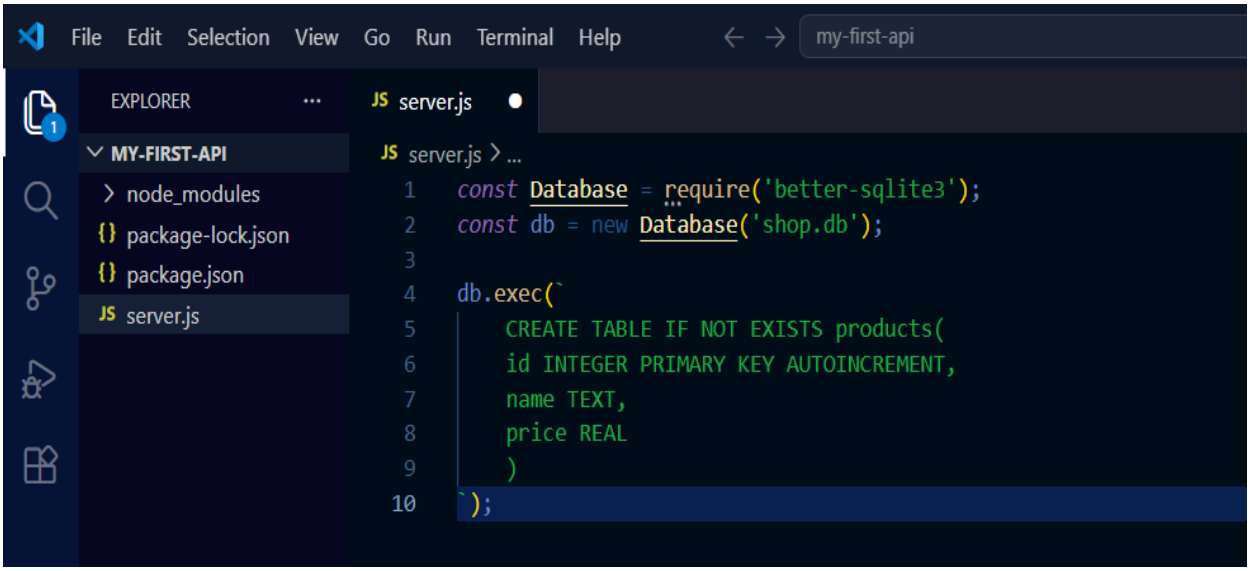
1. **Install the Database Driver:** In your terminal, inside the *my-first-api* folder, install the *better-sqlite3* package. This module acts as the **Local API** that allows your Node.js code to talk to a SQL database file.
    a. Command: *npm install better-sqlite3*



2. Initialize the Connection
    a. At the top of your *server.js*, you need to "require" the module and create a new database instance.
    b. *Hint:* Creating a *new Database('shop.db')* will automatically generate a file named *shop.db* in your project folder the moment the server starts.




3. Architecture
    a. Modify your *server.js*
        i. Table - Before your API can store anything, the database needs a "table." Use the *.exec()* method on your database object to run a SQL command that creates a table named *products* **if it doesn't already exist**.
            1. *Required Columns: id* (Primary Key, Autoincrement), *name* (Text), and *price* (Numeric).



ii. GET Endpoint (Fetch All)

1. Modify your *app.get('/api/products', ...)* route.
    a. Instead of sending a hardcoded message, you need to "Prepare" a SQL statement: *SELECT * FROM products.*
    b. Use the *.all()* method to retrieve every row from that table and send the resulting array back as a JSON response.
2. Modify your *app.get('/api/products', ...)* route.

```
app.get('/api/products', (req, res) =>{
    const stmt =db.prepare('SELECT * FROM products');
    const products = stmt.all();

    res.status(200).json(products);
});
```

iii. POST Endpoint (Insert Data)

1. Modify your *app.post('/api/products', ...)* route.
    a. Extract the *name* and *price* from the incoming *req.body.*
    b. Don't just inject strings. Use a "Prepared Statement" with placeholders (*?*) to prevent SQL injection.
    c. Run a SQL *INSERT INTO* command. Use the *.run()* method to execute it with the user's data.
    d. Send a *201* status and confirm the product was added.

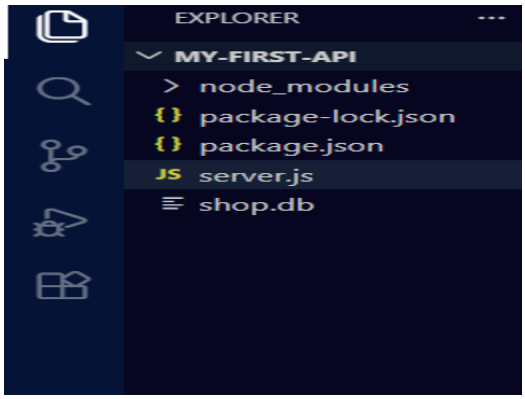
```
app.post('/api/products', (req, res) => {
    const {name, price} = req.body;

    const stmt = db.prepare('INSERT INTO products (name,price) VALUES (?, ?)');
    const result = stmt.run(name, price);

    res.status(201).json({
        message:"Product added successfully",
        id: result.lastInsertRowid
    });
});
```

4. Test Progress
   a. **Start the Server:** Run *node server.js*. Verify that a *shop.db* file appears in your file explorer.

```
PS C:\Users\emman\OneDrive\Documents\Setubal_BSIT3B_lab_4\my-first-api> node server.js
Server is running on http://localhost:3000
```

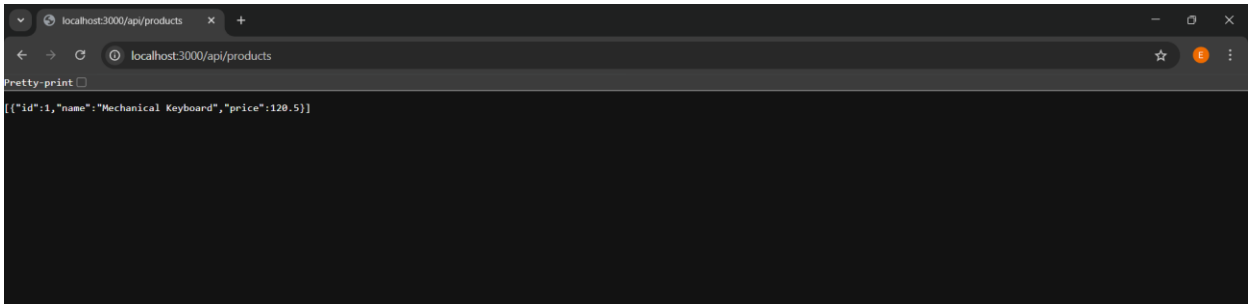



   b. **Add a Product (POST):** Use your terminal to send data
      i. Sample data using curl
         1. curl -X POST http://localhost:3000/api/products \
         2. -H "Content-Type: application/json" \
         3. -d '{"name": "Mechanical Keyboard", "price": 120.50}'

```
PS C:\Users\emman\OneDrive\Documents\Setubal_BSIT3B_lab_4\my-first-api> Invoke-RestMethod -Uri "http://localhost:3000/api/products" `
>> -Method POST `
>> -ContentType "application/json" `
>> -Body '{"name":"Mechanical Keyboard","price":120.50}'

message                   id
-------                   --
Product added successfully  1
```

   c. **Verify (GET):** Open *http://localhost:3000/api/products* in your browser.




   d. **The Persistence Test:** Stop your server (*Ctrl+C*). Start it again. Refresh the browser. If the keyboard is still there, you have successfully built a persistent API

5. CODE

EXPLORER

MY-FIRST-API

- node_modules
- package-lock.json
- package.json
- server.js
- shop.db

server.js > ...

```javascript
const express =require('express');
const Database = require('better-sqlite3');

const app =express();
const PORT = 3000;

app.use(express.json());


const db = new Database('shop.db');

db.exec(`
    CREATE TABLE IF NOT EXISTS products(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    price REAL
    )
`);

app.get('/api/greeting', (req, res) => {
    res.status(200).json({
        message: "Hello! Your API is working."
    });
});

app.post('/api/echo', (req, res) => {
    res.status(200).json(req.body);
});


app.get('/api/products', (req, res) =>{
    const stmt =db.prepare('SELECT * FROM products');
    const products = stmt.all();

    res.status(200).json(products);
});

app.post('/api/products', (req, res) => {
    const {name, price} = req.body;

    const stmt = db.prepare('INSERT INTO products (name,price) VALUES (?, ?)');
    const result = stmt.run(name, price);

    res.status(201).json({
        message:"Product added successfully",
        id: result.lastInsertRowid
    });
});

app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
});
```